

Министерство образования и науки Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 Математика и компьютерные науки

Отчет о выполнении курсовой работы
по дисциплине «Курсовое проектирование по управлению
ресурсами суперэвм»
«Решение задачи вычисления базисного минора и
алгебраического дополнения матрицы»

Выполнила студентка группы
№5130201/30101
Проверил

Лаишевкина А. М. _____
Курочкин М. А. _____

«_____» _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	4
1 Постановка задачи	5
2 Аппаратно-программная платформа Nvidia CUDA	6
2.1 Архитектура Nvidia CUDA	6
2.1.1 Архитектура Blackwell (март 2024)	6
2.1.2 Архитектура Horner (март 2022)	8
2.1.3 Архитектура Ады Лавлейс (сентябрь 2022)	9
2.1.4 Архитектура Ampere (2020)	10
2.1.5 Архитектура Pascal (2016)	12
2.1.6 Архитектура Maxwell (2014)	14
2.1.7 Архитектура Tesla (2006)	15
2.2 Вычислительные возможности Nvidia CUDA	17
2.3 Потокковая модель	18
2.4 Устройство памяти	19
2.5 Модели памяти	20
2.6 Модель вычислений на GPU	21
2.7 Компиляция программы	21
2.8 Планировщик задач	21
3 Суперкомпьютерный центр «Политехнический»	23
3.1 Состав	23
3.2 Характеристики	23
3.3 Технология подключения	23
4 Постановка решаемой практической задачи	25
5 Алгоритм решения задачи	26
5.1 Общий подход	26
5.2 Вычисление определителя методом LU-разложения	26
5.3 Метод распараллеливания алгоритма	26
5.3.1 Глобальная память	26
5.3.2 Разделяемая память	27
5.4 Вычисление базисного минора	27
6 Описание эксперимента	29
7 Анализ результатов	30
7.1 Результаты эксперимента	30
7.2 Зависимость времени от конфигурации (глобальная память)	34
7.3 Зависимость времени от конфигурации (разделяемая память)	35

Введение

Современные научные и инженерные задачи требуют значительных вычислительных мощностей для обработки больших объемов данных за приемлемое время. Одним из эффективных подходов к решению сложных вычислительных задач является применение технологий параллельного программирования.

Суперкомпьютеры представляют собой многопроцессорные вычислительные комплексы, способные выполнять триллионы операций в секунду. Их применение открывает возможности для решения задач, недоступных для традиционных последовательных вычислений. Это особенно актуально для задач линейной алгебры, численного моделирования, обработки больших массивов данных и машинного обучения.

В последние годы широкое распространение получили вычисления с использованием графических ускорителей — устройств с массивно-параллельной архитектурой. Производить вычисления общего назначения можно на видеокартах архитектуры NVIDIA CUDA, которая позволяет эффективно использовать тысячи вычислительных ядер для параллельной обработки данных.

Одним из примеров высокопроизводительных вычислительных систем является суперкомпьютерный центр «Политехнический». Часть узлов этого суперкомпьютера оборудована графическими ускорителями NVIDIA Tesla K40X, что позволяет решать широкий класс задач линейной алгебры с высокой эффективностью.

Задачи вычисления базисных миноров и алгебраических дополнений матриц являются фундаментальными в линейной алгебре и находят применение при решении систем линейных уравнений, вычислении обратных матриц, определении ранга матрицы и в других областях вычислительной математики.

1 Постановка задачи

Целью курсовой работы является исследование технологий параллельного программирования на основе архитектуры NVIDIA CUDA для решения задачи вычисления базисного минора и алгебраического дополнения матрицы с использованием вычислительных ресурсов СКЦ «Политехнический».

Для достижения поставленной цели необходимо решить следующие задачи:

- Изучить аппаратно-программную платформу NVIDIA CUDA: её архитектуру, вычислительные возможности, устройство памяти, модель вычислений.
- Изучить состав и характеристики вычислительных средств СКЦ «Политехнический», изучить и применить технологию подключения к СКЦ.
- Реализовать с использованием технологии CUDA C алгоритм вычисления базисного минора и алгебраического дополнения матрицы.
- Провести серию вычислительных экспериментов на узле «Торнадо» СКЦ «Политехнический» с изменением следующих параметров:
 - количество потоков в блоке,
 - количество блоков,
 - размер исходной матрицы,
 - используемые модели памяти.
- Провести анализ полученных результатов и сравнить время выполнения реализации при различных параметрах.

2 Аппаратно-программная платформа Nvidia CUDA

NVIDIA CUDA (Compute Unified Device Architecture) — это программная и аппаратная платформа от NVIDIA, которая позволяет использовать графические процессоры (GPU) для параллельных вычислений и ускорения обработки данных.

2.1 Архитектура Nvidia CUDA

Архитектура GPU включает:

- Процессор GPU, который представляет собой массив потоковых процессоров (Streaming Processor Array);
- Потоковый процессор, который состоит из кластеров текстурных процессоров (Texture Processor Clusters);
- Текстурные процессоры, который состоят из набора мультипроцессоров (Streaming Multiprocessor);
- Мультипроцессоры, которые содержат несколько потоковых процессоров (Streaming Processors) или ядер (CUDA cores).

Планирование выполнения команд происходит при помощи GigaThread Engine. Он распределяет блоки потоков по мультипроцессорам. Общий вид GPU кардинально не меняется при переходе от одной микроархитектуры к другой, изменяется размер и скорость L2 кэша.

В архитектуре Nvidia CUDA применяется SIMT (Single Instruction Multiple Thread) модель исполнения. Эта модель является комбинацией из MIMD (Multiple Instruction Multiple Data) и SIMD (Single Instruction Multiple Data). Задача разбивается на множество блоков. Планировщик распределяет блоки по SM (вычислительным блокам), при этом каждый блок выполняется только на одном SM, но на одном SM может выполняться несколько блоков.

Каждый блок состоит из потоков (threads), при исполнении потоки разбиваются на варпы — группы по 32 потока. Все потоки варпа выполняют в одно время одну и ту же инструкцию.

2.1.1 Архитектура Blackwell (март 2024)

Архитектура Blackwell представляет собой значительный шаг вперед в развитии графических технологий от NVIDIA, охватывая три ключевых аспекта применения: гейминг, студийные задачи и искусственный интеллект. Рассмотрим подробно каждый аспект, подчеркивая уникальные технологии и преимущества архитектуры.

Видеокарты серии Blackwell уже доступны в продаже. Надёжным партнёром по их поставке выступает интернет-магазин Telemart, предлагающий широкий ассортимент моделей NVIDIA GeForce RTX 50-й серии, включая как игровые, так и профессиональные решения.

Архитектура Blackwell делает особый акцент на реалистичную визуализацию человеческой кожи в играх. Технология RTX Skin использует усовершенствованные алгоритмы подповерхностного рассеивания света (SSS), что позволяет добиться правдоподобной передачи полутонов, текстур и цветовых переходов. В отличие от предыдущих реализаций, система теперь учитывает не только основной свет, но и контекстные изменения освещения — в том числе влияние окружающей среды и отражённого света от других поверхностей.

Особое внимание уделено адаптивным алгоритмам, которые в реальном времени настраивают интенсивность и глубину рассеивания света под кожей. Это позволяет сохранить корректное отображение даже при резкой смене условий — например, при переходе из солнечного пространства в тень или при использовании источников холодного направленного света. Эффект особенно заметен в сценах с близкими планами, где важны микродетали текстуры кожи — поры, сосуды, тональные переходы.

Первое практическое применение RTX Skin можно увидеть в Half-Life 2 RTX Remaster, где технология используется для обновлённого отображения лиц главных персонажей. В катсценах и диалогах заметно, как кожа реагирует на освещение от фонарей, солнечного света или светящихся экранов. Такой уровень фотореализма делает старую игру визуально современнее и подчёркивает потенциал RTX Skin в кинематографичных проектах нового поколения.

DLSS 4 с Frame Generation и Multi-Frame Generation

DLSS 4 представляет собой масштабное обновление технологии Deep Learning Super Sampling от NVIDIA, анонсированное одновременно с архитектурой Blackwell и видеокартами серии GeForce RTX 50. Основными нововведениями стали использование модели трансформера и интеграция Multi Frame Generation, благодаря которым достигается значительный прирост производительности и резкое улучшение визуального качества — особенно в динамичных сценах и при высоких разрешениях.

Новая система генерации кадров теперь способна учитывать не только текущий и предыдущий кадры, но и пространственно-временные зависимости между несколькими отрезками рендеринга, что минимизирует артефакты и повышает чёткость изображения. DLSS 4 также задействует более сложную нейросетевую архитектуру, позволяющую адаптироваться к конкретной игре и сценарию, обеспечивая более стабильный результат.

2.1.2 Архитектура Norper (март 2022)

Архитектура Норпер представляет собой высокопараллельную вычислительную платформу, спроектированную на базе графического процессора GH100. Структура архитектуры опирается на сложную иерархическую организацию вычислительных блоков и глубоко интегрированную подсистему памяти, ориентированную на максимизацию локальности данных.

Вычислительное ядро и потоковые мультипроцессоры (SM) Базовым строительным блоком архитектуры выступает потоковый мультипроцессор (Streaming Multiprocessor, SM). Каждый SM содержит независимые конвейеры для целочисленных вычислений (INT32) и вычислений с плавающей запятой, а также специализированные блоки матричной математики.

- **Тензорные ядра (Tensor Cores):** Вычислительные блоки, аппаратно реализующие операции умножения и сложения матриц (Matrix Multiply-Accumulate). Они аппаратно поддерживают вычисления смешанной точности, включая форматы FP8, FP16, bfloat16, TF32 и FP64.
- **Transformer Engine:** Аппаратный модуль, функционирующий совместно с тензорными ядрами на уровне SM. Он выполняет эвристический анализ распределения значений в тензорах на каждом слое нейронной сети и осуществляет динамическое (на лету) переключение точности вычислений между форматами FP8 и FP16, оптимизируя информационную энтропию и пропускную способность.
- **Инструкции DPX:** Интегрированный в арифметико-логические устройства (ALU) набор инструкций, реализующий аппаратное ускорение рекурсивных алгоритмов динамического программирования на уровне отдельных тактов процессора.

Иерархия памяти и асинхронная передача данных

Для предотвращения простоя вычислительных блоков (data starvation), архитектура Норпер включает специализированные механизмы управления памятью:

- **Тензорный ускоритель памяти (Tensor Memory Accelerator, TMA):** Выделенный аппаратный блок управления прямым доступом к памяти. TMA берет на себя задачи асинхронного копирования многомерных блоков данных между глобальной памятью (HBM3) и разделяемой памятью (Shared Memory). Он вычисляет адреса и управляет транзакциями аппаратно, полностью высвобождая регистры и ALU потокового мультипроцессора.
- **Распределенная разделяемая память (Distributed Shared Memory, DSMEM):** Архитектурная инновация, позволяющая объединять пулы

локальной разделяемой памяти нескольких SM. Мультипроцессоры могут напрямую читать и писать данные в L1/Shared память соседних SM по выделенному высокоскоростному интерконнекту, минуя обращение к кэшу второго уровня (L2) или глобальной памяти.

Иерархия планирования: Thread Block Clusters

В архитектуре Horner традиционная иерархия планирования потоков (Grid → Block → Thread) расширена новым промежуточным уровнем абстракции — кластерами блоков потоков (*Thread Block Clusters*). Эта парадигма позволяет группировать несколько блоков потоков на этапе компиляции и аппаратно гарантировать их совместное (кооперативное) планирование на физически соседних SM. Потоки внутри одного кластера получают возможность аппаратной барьерной синхронизации и прямого использования технологии DSMEM, что позволяет обрабатывать крупные фрагменты графов вычислений локально, не выгружая промежуточные результаты в общую память устройства.

2.1.3 Архитектура Ады Лавлейс (сентябрь 2022)

Архитектура Ada Lovelace представляет собой вычислительную платформу, аппаратно оптимизированную для ресурсоемкого графического рендеринга, симуляции физики света и нейросетевого инференса. Структурная парадигма архитектуры строится на глубокой специализации вычислительных блоков в рамках потоковых мультипроцессоров и оптимизации путей передачи данных.

Вычислительное ядро и специализированные сопроцессоры Фундаментальным вычислительным узлом является потоковый мультипроцессор (SM), объединяющий блоки общего и узкоспециализированного назначения:

- **Ядра CUDA (Универсальные ALU):** Блоки, реализующие архитектуру параллельных вычислений и обеспечивающие одновременное (конкурентное) выполнение инструкций с плавающей запятой (FP32) и целочисленных операций (INT32).
- **Тензорные ядра (Tensor Cores):** Аппаратные матричные сопроцессоры, адаптированные для вывода нейросетей и обработки алгоритмов искусственного интеллекта. Они поддерживают расширенный спектр форматов данных, включая тензорный формат FP8, что позволяет значительно повысить пропускную способность при сохранении необходимой точности в задачах глубокого обучения.
- **Ядра трассировки лучей (RT Cores):** Выделенные аппаратные блоки для расчетов пересечения лучей с ограничивающими объемами (BVH) и треугольниками. Включают интегрированные подмодули: *Opacity Micromap*

Engine для аппаратного исключения расчетов прозрачной геометрии (например, листвы) и *Displaced Micro-Mesh Engine* для генерации высокодетализированной геометрии на аппаратном уровне без перегрузки основных конвейеров и подсистемы памяти.

Аппаратная перекомпоновка и медиа-движки

Для борьбы с нерегулярными вычислительными нагрузками в архитектуру внедрены специализированные аппаратные планировщики:

- **Shader Execution Reordering (SER):** Аппаратный механизм динамического переупорядочивания потоков (threads) на лету. В задачах с высокой дивергенцией (когда лучи отражаются в разных направлениях и требуют разных шейдеров), SER группирует потоки с аналогичными инструкциями или схожим доступом к памяти. Это превращает нерегулярную нагрузку в последовательную, максимизируя утилизацию вычислительных блоков мультипроцессора.
- **Ускоритель оптического потока (Optical Flow Accelerator, OFA):** Выделенный аппаратный блок машинного зрения, который анализирует последовательные кадры и вычисляет векторы движения пикселей. Опираясь на эти данные, тензорные ядра могут генерировать полностью новые кадры методами ИИ, минуя традиционный конвейер рендеринга.
- **Двойные аппаратные кодировщики (Dual NVENC):** Интегрированные медиа-процессоры с аппаратной поддержкой кодирования и декодирования формата AV1, способные разделять кадр на части для параллельной обработки.

Подсистема памяти и локализация данных

Архитектура памяти спроектирована с упором на максимальное удержание данных внутри чипа:

- **Масштабированный кэш второго уровня (L2 Cache):** В архитектуре реализован радикально увеличенный объем унифицированной кэш-памяти L2. Это структурное решение гарантирует, что массивы данных BVH, необходимые для трассировки лучей, и весовые коэффициенты нейросетей остаются в высокоскоростной памяти графического процессора. Это минимизирует количество транзакций по шине к подсистеме видеопамяти (VRAM), снижая энергопотребление и задержки (latency) при доступе к данным.

2.1.4 Архитектура Ampere (2020)

Архитектура Ampere представляет собой унифицированную платформу, архитектурно сбалансированную для удовлетворения требований как традиционных графических рабочих нагрузок, так и интенсивных вычислений в

области искусственного интеллекта (ИИ) и высокопроизводительных вычислений (HPC). Основной фокус архитектуры — повышение энергоэффективности, параллелизма и введение новых уровней точности вычислений.

Вычислительное ядро и потоковые мультипроцессоры (SM) Поточковый мультипроцессор (SM) в архитектуре Ampere подвергся значительной переработке для обеспечения большей гибкости конвейеров:

- **Разделенные пути данных (Datapaths):** Архитектура внедряет параллельные конвейеры для выполнения инструкций. Один путь выделен исключительно для операций с плавающей запятой (FP32), в то время как второй может динамически переключаться между целочисленными операциями (INT32) и операциями FP32 в пределах одного такта. Это удваивает теоретическую пиковую производительность FP32 по сравнению с архитектурами предыдущих поколений.
- **Тензорные ядра (Tensor Cores):** Аппаратные блоки матричного умножения оснащены поддержкой новых форматов данных: *TF32* (*Tensor Float 32*) и *Bfloat16*. Формат TF32 позволяет обрабатывать данные FP32 с диапазоном FP32, но с точностью FP16, что обеспечивает значительное ускорение обучения нейросетей без модификации программного кода.
- **Механизм структурной разреженности (Structural Sparsity):** Тензорные ядра получили аппаратную поддержку разреженных матриц с соотношением 2:4 (на каждые четыре значения в векторе аппаратура ожидает два нулевых). Оборудование автоматически сжимает данные и пропускает вычисления с нулями, удваивая математическую пропускную способность и снижая требования к пропускной способности памяти при инференсе нейронных сетей.
- **Ядра трассировки лучей (RT Cores):** Аппаратные блоки, выполняющие параллельные вычисления пересечений лучей с треугольниками и ограничивающими объемами (BVH). В архитектуре реализовано одновременное выполнение вычислений трассировки лучей, операций рендеринга (затенения) и тензорных вычислений.

Иерархия памяти и управление данными

Подсистема памяти Ampere спроектирована для обеспечения непрерывного потока данных к высокопроизводительным SM:

- **Асинхронное копирование (Asynchronous Copy):** На уровне SM реализованы новые инструкции для асинхронного копирования данных непосредственно из глобальной памяти в разделяемую память (Shared Memory), минуя регистровый файл. Это высвобождает регистры и снижает нагрузку с ALU во время транзакций памяти.

- **L2 Cache Residence Controls:** Архитектура предоставляет программируемые элементы управления на уровне кэша L2. Это позволяет явно назначать фрагментам данных (например, часто используемым матрицам или структурам BVH) приоритет для постоянного хранения (residency) в кэше L2, минимизируя дорогостоящие обращения к глобальной памяти.

Мультиарендность и виртуализация

Для корпоративных сред и центров обработки данных в архитектуру внедрен аппаратный механизм разделения ресурсов:

- **Multi-Instance GPU (MIG):** Аппаратная технология, позволяющая изолировать и разбивать один физический графический процессор на несколько (до семи) независимых инстансов. Каждый инстанс получает гарантированные и полностью изолированные на аппаратном уровне ресурсы: сегменты кэша L2, пропускную способность памяти, потоковые мультипроцессоры и блоки кодирования. Это обеспечивает строгую изоляцию отказов (fault isolation) и предсказуемую производительность при параллельном выполнении разнородных задач.

2.1.5 Архитектура Pascal (2016)

Архитектура Pascal представляет собой фундаментальный этап в эволюции графических процессоров, ознаменовавший переход к аппаратному ускорению высокопроизводительных вычислений (HPC) и алгоритмов глубокого обучения. В ее основе лежат радикальные изменения в организации памяти, коммуникационных интерфейсах и вычислительной точности.

Вычислительное ядро и потоковые мультипроцессоры (SM) Потоковый мультипроцессор в архитектуре Pascal (в частности, в старшем ядре GP100) был структурно реорганизован для повышения эффективности планирования инструкций и максимизации утилизации регистров:

- **Оптимизированная структура SM:** Мультипроцессор разделен на два независимых вычислительных блока (вместо четырех в архитектуре Maxwell), каждый из которых имеет свой собственный буфер инструкций, планировщик (warp scheduler) и диспетчер. Это упрощает логику диспетчеризации и увеличивает долю площади кристалла, отводимой непосредственно под арифметико-логические устройства (ALU) и регистровый файл.
- **Аппаратная поддержка смешанной точности (Mixed-Precision Computing):** Архитектура впервые внедрила нативную аппаратную поддержку вычислений с половинной точностью (FP16). Инструкции FP16 могут упаковываться попарно (векторизация) и выполняться на ALU одинарной точности, что позволяет удвоить пропускную способность по срав-

нению с FP32. Это стало критически важным механизмом для ускорения обучения нейронных сетей.

Инновации в подсистеме памяти и адресации

Для преодоления ограничений пропускной способности (решения проблемы «стены памяти») архитектура Pascal использует передовые технологии физической компоновки и виртуализации данных:

- **Память HBM2 (High Bandwidth Memory):** Использование технологии 2.5D компоновки, при которой многослойные стеки памяти размещаются на одной кремниевой подложке (interposer) с графическим процессором. Это решение позволило многократно увеличить пропускную способность памяти (до 720 ГБ/с в первых реализациях) и существенно снизить задержки по сравнению с дискретной памятью GDDR5.
- **Унифицированная память и аппаратный страничный обмен (Page Migration Engine):** Внедрение выделенного аппаратного контроллера, поддерживающего обработку ошибок отсутствия страницы памяти (page faults) со стороны GPU. Это обеспечило создание истинно единого виртуального адресного пространства между CPU и GPU. При обращении графического процессора к данным, находящимся в оперативной памяти хоста, контроллер автоматически, страница за страницей, мигрирует необходимые данные в локальную память VRAM по требованию (on-demand), абстрагируя программиста от ручного управления пересылками.

Механизмы коммуникации и планирования задач

Архитектура пересмотрела подходы к масштабированию многопроцессорных систем и разделению вычислительных ресурсов:

- **Высокоскоростной интерконнект NVLink:** Интеграция первого поколения проприетарной шины для когерентного и прямого взаимодействия между графическими процессорами (GPU-to-GPU), минуя корневой комплекс PCIe. NVLink обеспечил радикальное расширение полосы пропускания обмена данными, что является обязательным условием для эффективного распределенного обучения моделей (model parallelism) в кластерах.
- **Вытесняющая многозадачность (Compute/Instruction-Level Preemption)**
Аппаратный механизм тонкогранулярного планирования, позволяющий прерывать выполнение вычислительных задач (compute kernels) на уровне отдельных инструкций процессора. При прерывании GPU аппаратно сохраняет полный контекст выполнения задачи в память, уступает ресурсы более приоритетному процессу, а затем восстанавливает контекст и продолжает работу. Это гарантирует предсказуемую задержку отклика системы и устраняет проблему монополизации мультипроцессоров длительными ядрами.

2.1.6 Архитектура Maxwell (2014)

Архитектура Maxwell стала знаковым этапом в развитии графических процессоров NVIDIA, сместив акцент с простого наращивания количества вычислительных блоков на радикальное повышение энергетической эффективности и оптимизацию использования ресурсов каждого транзистора. Основные инновации сосредоточены в области внутренней логики планирования и глубокой модернизации подсистемы памяти.

Реорганизация потоковых мультипроцессоров (SMM) В архитектуре Maxwell была представлена новая структура мультипроцессора — SMM (Maxwell Streaming Multiprocessor), разработанная для достижения максимальной производительности на ватт:

- **Секционирование и логика планирования:** Каждый SMM разделен на четыре отдельных блока обработки, каждый из которых обладает собственным буфером инструкций, планировщиком (warp scheduler) и диспетчером. В отличие от предыдущих итераций, такая жесткая привязка ресурсов к конкретным группам CUDA-ядер позволила упростить логику управления и существенно снизить энергопотребление накладных расходов планирования.
- **Эффективность CUDA-ядер:** Несмотря на меньшее количество ядер на один SM (128 против 192 в Kepler), каждый SMM обеспечивает значительно более высокую полезную нагрузку за счет оптимизации путей передачи инструкций и улучшенного заполнения конвейеров, что позволяет одному ядру Maxwell выполнять до 1.35 раза больше работы при той же частоте.
- **Разделяемая память и кэш L1:** В SMM разделяемая память (Shared Memory) была физически отделена от кэша L1, что увеличило доступный объем ресурсов для высокопараллельных задач и устранило конфликты при распределении памяти, характерные для комбинированных блоков.

Оптимизация подсистемы памяти и сжатия данных

Для эффективной работы в условиях ограниченной ширины шины памяти, архитектура Maxwell внедряет передовые методы сокращения избыточности данных:

- **Третье поколение дельта-сжатия цвета (Delta Color Compression):** Графический процессор получил аппаратную поддержку более эффективных алгоритмов сжатия без потерь. Это позволяет записывать в видеопамять только разницу (дельту) между значениями соседних пикселей, что эффективно увеличивает пропускную способность памяти до 25-30% за счет снижения объема передаваемого трафика.

- **Масштабированный кэш второго уровня (L2 Cache):** В архитектуре реализовано значительное увеличение объема кэша L2 (до 2 МБ в чипах среднего сегмента, что в 8 раз больше предшественников). Это позволяет удерживать значительно больше рабочих данных на кристалле, сокращая количество энергозатратных обращений к внешней памяти VRAM.

Специализированные аппаратные блоки и алгоритмы освещения

Архитектура Maxwell привнесла новые возможности для вычислений в реальном времени и обработки медиаконтента:

- **Voxel Global Illumination (VXGI):** Аппаратная поддержка алгоритмов воксельного глобального освещения. Процессор способен эффективно переводить геометрию сцены в воксельную структуру для быстрого просчета отраженного света и мягких теней.
- **Обновленный NVENC:** Интеграция выделенного кодировщика с поддержкой формата H.265 (HEVC). Это позволило аппаратно обрабатывать видео сверхвысокого разрешения (4K) с минимальными задержками и без нагрузки на основные вычислительные CUDA-ядра.
- **Multi-Pixel Programming Sampling:** Технология, оптимизирующая работу блоков растеризации для поддержки новых режимов сглаживания и повышения качества выборки пикселей при рендеринге сложных сцен.

2.1.7 Архитектура Tesla (2006)

Архитектура Tesla представляет собой первую в индустрии полностью унифицированную вычислительную архитектуру графических процессоров. Главным концептуальным сдвигом стал отказ от отдельных аппаратных блоков для обработки вершинных и пиксельных шейдеров в пользу массива универсальных скалярных процессоров, способных выполнять любой тип вычислительной нагрузки.

Унифицированная архитектура и скалярная обработка В основе Tesla лежит отказ от традиционной векторной обработки данных в пользу мелкозернистой скалярной архитектуры:

- **Унифицированные шейдерные ядра:** Вместо выделенных аппаратных ресурсов для разных стадий графического конвейера, Tesla использует единый массив потоковых процессоров (Streaming Processors, SP). Это позволяет динамически перераспределять вычислительную мощность в зависимости от текущей нагрузки, устраняя простои оборудования.

- **Скалярное исполнение инструкций:** В отличие от предыдущих архитектур, оперировавших векторами (например, RGBA), процессоры Tesla работают со скалярными величинами. Это упрощает компиляцию кода и повышает эффективность использования ресурсов, так как системе не требуется заполнять все компоненты вектора для выполнения операции.

Организация потоковых мультипроцессоров (SM)

Вычислительные ресурсы Tesla сгруппированы в иерархическую структуру, оптимизированную для параллельного выполнения тысяч потоков:

- **Потоковый мультипроцессор (SM):** Каждый SM в архитектуре Tesla содержит 8 скалярных процессоров (SP), два специальных функциональных блока (SFU) для вычисления трансцендентных функций и блок управления потоками.
- **Разделяемая память (Shared Memory):** Важнейшим нововведением стало появление локальной памяти объемом 16 КБ внутри каждого SM. Эта память имеет крайне низкие задержки и позволяет потокам внутри одного блока обмениваться данными напрямую, что фактически превратило GPU в полноценный параллельный процессор.
- **Механизм выполнения Warps:** Архитектура ввела понятие «варпа» (warp) — группы из 32 потоков, которые выполняют одну и ту же инструкцию одновременно (SIMT — Single Instruction, Multiple Threads). Это позволило скрыть задержки обращения к памяти за счет мгновенного переключения между активными группами потоков на аппаратном уровне.

Подсистема памяти и модель ввода-вывода

Для поддержки общих вычислений Tesla внедрила новые механизмы взаимодействия с системными ресурсами:

- **Глобальное адресное пространство:** Tesla обеспечила возможность прямой адресации памяти, что позволило реализовать поддержку языков программирования высокого уровня, таких как C, через программно-аппаратный стек CUDA.
- **Текстурный и константный кэши:** Несмотря на универсальность, архитектура сохранила специализированные кэши для чтения данных, оптимизированные под конкретные паттерны доступа (например, двумерную локальность в текстурах), что существенно ускоряет работу графических и научных приложений.

2.2 Вычислительные возможности Nvidia CUDA

Видеокарты Nvidia CUDA разных микроархитектур обладают разным количеством ядер разного назначения. Ниже приведены краткие сведения о вычислительных возможностях различных поколений Nvidia CUDA.

Микроархитектура Fermi

- В каждом потоковом процессоре (SM):
 - 32 CUDA Core (ядра CUDA для выполнения операций с целыми числами и с числами с плавающей точкой);
 - 16 LD/ST блоков (ядер загрузки/выгрузки данных);
 - 4 SFU — блока специального назначения (для вычисления сложных арифметических функций).
- Планировщики: 2 Warp Scheduler с dual-issue (параллельное выполнение независимых инструкций).
- Память: регистровый файл, 128KB; L1-cache, 16KB/48KB; разделяемая память, 48KB/16KB.
- Особенности: базовая структура SM, поддержка dual-issue для повышения производительности.

Микроархитектура Kepler

- SMX: 192 CUDA Core (12 групп по 16 ядер), 64 блока двойной точности (FP64), 32 LD/ST, 32 SFU.
- Планировщики: 4 Warp Scheduler с двумя блоками отправки команд каждый.
- Память: регистровый файл, 256KB; L1-cache, 16KB/48KB/32KB; разделяемая память, 48KB/16KB/32KB; константная память (Read-Only Data Cache), 48KB.

Микроархитектура Maxwell

- SMM: 4 блока по 32 CUDA Core, 8 LD/ST блоков, 8 SFU блока.
- Память: 64KB регистровый файл на блок, 90KB Shared Memory (отдельный блок).
- Особенности: модульная структура, улучшение производительности на ватт в 2 раза, на площадь кристалла — в 1.4 раза.

Микроархитектура Pascal

- SM: 2 блока с удвоенным регистровым файлом и увеличенным числом блоков двойной точности.
- Память: 64KB Shared Memory, Texture/L1 кэш.
- Особенности: переход на 16нм FinFET, улучшение производительности на ватт.

Микроархитектура Turing

- SM: 4 блока, каждый содержит 32 FP32 ядра, 32 INT ядра, 8 FP64 ядер, 2 Tensor Core, 8 LD/ST, 1 SFU.
- Tensor Core: обработка матриц 4×4 (FP16/FP32 вход, FP32/FP64 выход), прирост до 9.3x в задачах глубокого обучения.
- Память: 128KB L1/Shared Memory (до 96KB для Shared Memory).
- Особенности: Independent Thread Scheduling (у каждого потока свой счетчик команд), 12нм FinFET, Tensor Core для ИИ.

2.3 Потокковая модель

Вычислительная архитектура CUDA основана на концепции мультипроцессора и SIMT (Single Instruction Multiple Threads) — одна команда на множество данных.

При выполнении многопоточной программы на видеокарте CUDA, все потоки разделяются на блоки, а внутри блоков на варпы, где все потоки выполняют одну и ту же инструкцию. Группа блоков выполняется на потоковом процессоре, распределением задач занимается планировщик.

Программа, выполняющаяся на нескольких блоках одновременно называется ядром (kernel).

Особенностью архитектуры CUDA является блочно-сеточная организация, необычная для многопоточных приложений. Данная организация представлена на рисунке 1. При этом драйвер CUDA самостоятельно распределяет ресурсы устройства между потоками.

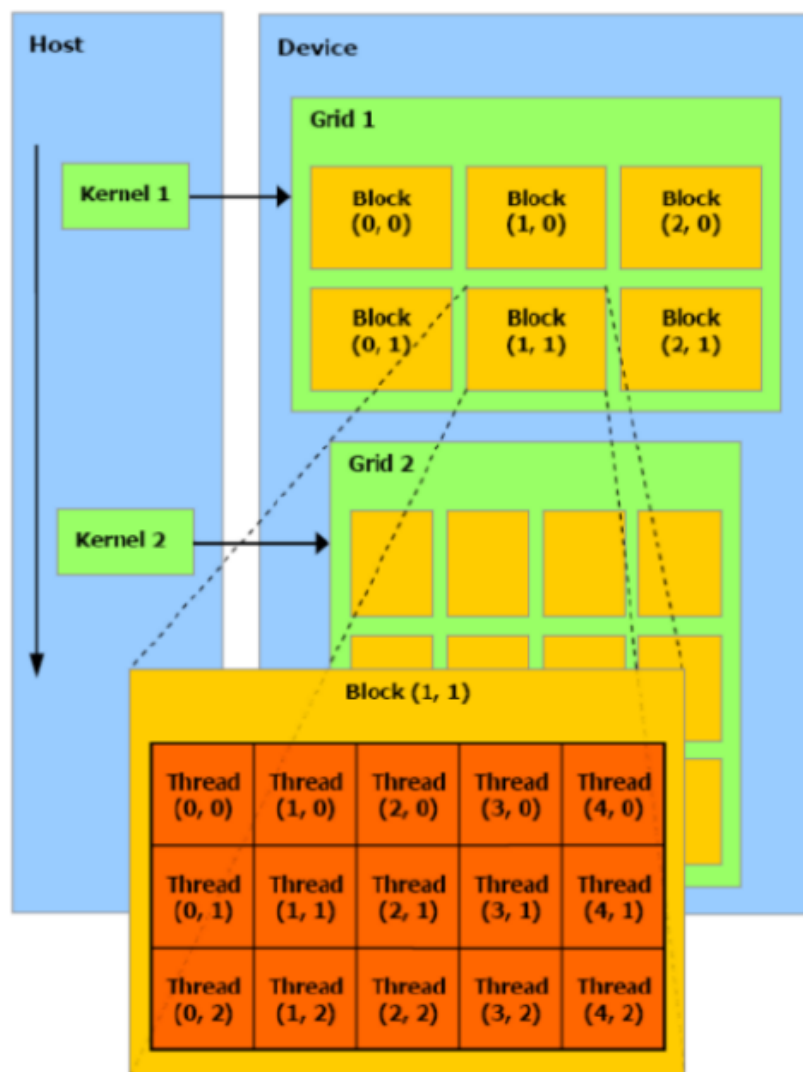


Рис. 1. Организация потоков

2.4 Устройство памяти

Видеокарта имеет собственную оперативную глобальную память, отдельную от оперативной памяти CPU.

Когда на видеокарте выполняется код, доступ возможен только к её собственной памяти. Поэтому для выполнения вычислений и получения результата необходимо перенести данные из оперативной памяти ЦП в память видеокарты и обратно. Помимо глобальной оперативной памяти на каждом мультимикропроцессоре есть свой кэш, своя текстурная, константная, разделяемая память и т.д. Схема организации памяти CUDA-программы представлена на рисунке 2.

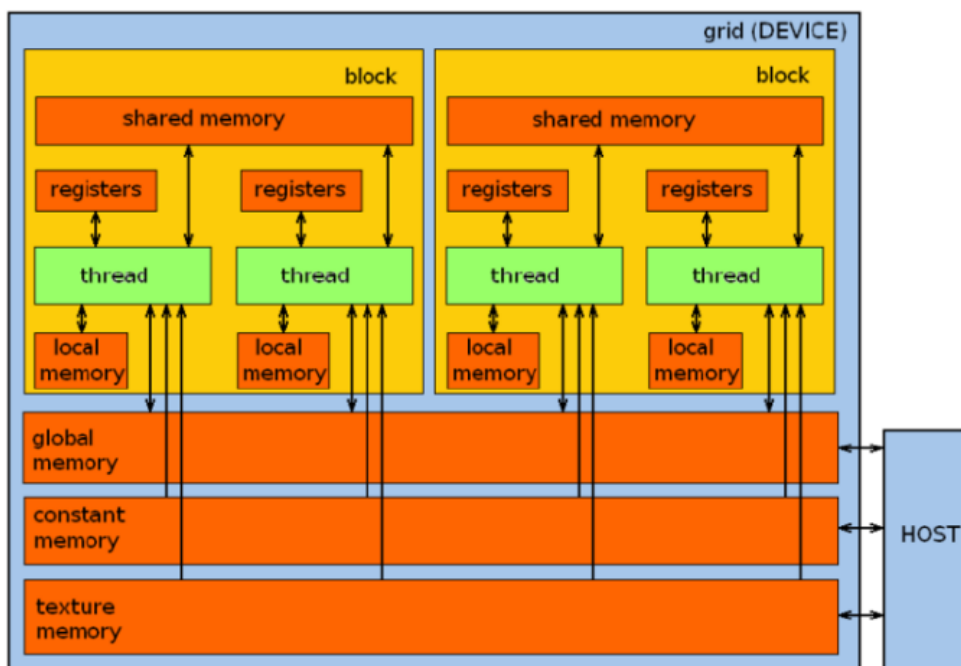


Рис. 2. Схема организация памяти в CUDA-программе

2.5 Модели памяти

В CUDA выделяют шесть видов памяти:

- **Регистры**

Это память, в которой по возможности компилятор старается размещать все локальные переменные функций. На один мультипроцессор доступно 8192 32-разрядных регистров. Они распределяются между нитями в этом потоке. Обращение к этой памяти самое быстрое.

- **Локальная память**

Это небольшой объём памяти, к которому имеет доступ только один потоковый процессор. В этот тип памяти помещаются локальные данные, которые имеют слишком большой размер. Физически локальная память является аналогом глобальной памяти, и работает с той же скоростью.

- **Глобальная память**

Глобальная память имеет большой объем (размер от 256 Мбайт до 4 Гбайт). Она поддерживает произвольный доступ для всех мультипроцессоров, а также запись и чтение с хоста. Однако, эта память очень медленная и не кэшируется, поэтому рекомендуется сократить количество обращений к этой памяти.

- **Разделяемая память**

Разделяемая память — это некешируемая, но быстрая память. Её рекомендуется использовать в качестве управляемого кэша. На один мультипроцессор доступно всего 16KB разделяемой памяти. Она обеспечивает взаимодействие потоков, управляется разработчиком напрямую и имеет низкие задержки.

- **Константная память**

Константная память — это тип памяти, который хранит неизменяемые данные, доступные на уровне сетки. Её размер 64KB.

- **Текстурная память**

Текстурная память — это тип памяти, который похож на константную память, поскольку через текстурный блок разрешены запросы только для чтения.

2.6 Модель вычислений на GPU

Программа, запускаемая на GPU с Nvidia CUDA, называется ядром (kernel). Ядро запускается одновременно на сетке из блоков. Каждый из блоков состоит из нескольких нитей. Количество блоков и потоков задаётся при вызове ядра. В каждой нити выполняется один и тот же код.

Добиться разного поведения в нитях возможно при помощи информации о номере блока и нити, которая доступна для каждой нити.

2.7 Компиляция программы

Программа для видеокарт Nvidia CUDA пишется на основе других языков, в частности используется расширение языка C. Оно называется CUDA C.

Для сборки программы используется компилятор `nvcc`, который входит в пакет инструментов разработчика. Этот пакет, а также библиотеку CUDA можно скачать с сайта Nvidia.

2.8 Планировщик задач

Поток (stream) в CUDA — логическая последовательность зависимых асинхронных операций, независимая от операций в других потоках.

Потоки позволяют запускать CUDA команды на GPU в порядке, определённом в контексте одного потока.

Планировщик задач в CUDA — это механизм, который управляет выполнением потоков на GPU. Он отвечает за распределение ресурсов, таких как вычислительные ядра и память, между потоками и блоками.

Kepler SM (SMX)

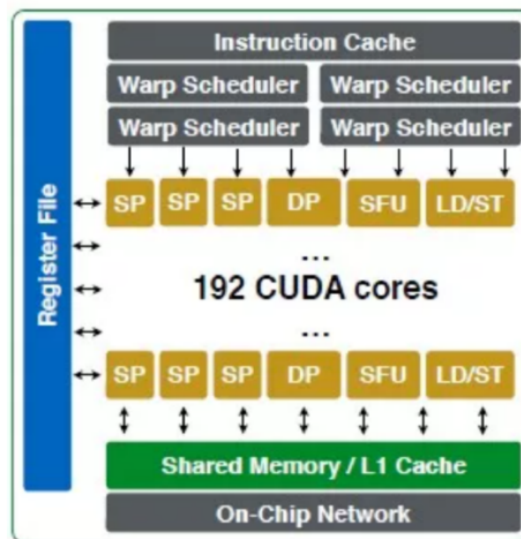


Рис. 3. Планировщик задач в архитектуре Kepler

3 Суперкомпьютерный центр «Политехнический»

3.1 Состав

Пользователям СКЦ «Политехнический» в настоящее время доступны следующие вычислительные ресурсы:

- 612 узлов кластера «Политехник – РСК Торнадо»;
- 56 узлов кластера «Политехник – РСК Торнадо» с ускорителями NVIDIA K-40;
- 288 узлов вычислителя с ультравысокой многопоточностью «Политехник – РСК ПетаСтрим».

3.2 Характеристики

- один узел кластера «Политехник – РСК Торнадо» содержит:
 - 2 x Intel Xeon CPU E5-2697 v3 @ 2.60GHz
 - 64G RAM
- один узел кластера «Политехник – РСК Торнадо» с ускорителями NVIDIA K-40 содержит:
 - 2 x Intel Xeon CPU E5-2697 v3 @ 2.60GHz
 - 64G RAM
 - 2 x Nvidia Tesla K40x 12G GDDR
- один узел кластера «Политехник – РСК ПетаСтрим» содержит:
 - 1 x Intel Xeon Phi 5120D @ 1.10GHz
 - 8G RAM

Все доступные узлы используют сеть 56Gbps FDR InfiniBand в качестве интерконнекта. Также, на всех узлах доступна параллельная файловая система Lustre объёмом около 1 ПБ.

По умолчанию пользователю предоставляется доступ к узлам tornado. Доступ к остальным типам узлов предоставляется по запросу.

3.3 Технология подключения

Для подключения к СКЦ «Политехнический» были выполнены следующие шаги:

1. От администрации СКЦ были получены публичный и приватный ключи в виде файлов, а также логин пользователя для доступа к системе.
2. Установка программного обеспечения:

На ноутбуке с операционной системой Windows 11 OpenSSH предустановлен, однако нужно запустить вручную службу ssh-agent. для этого нужно запустить cmd от имени администратора и выполнить команды

Далее нужно

3. Для подключения к узлу СКЦ через SSH в терминале PowerShell была введена следующая команда

где:

В результате осуществлено подключение к СКЦ по протоколу SSH. На рисунке 5 представлена стартовая страница при подключении к СКЦ.

Рис. 4. Подключение к СКЦ

4 Постановка решаемой практической задачи

Дано:

- Матрица M размером $n \times n$, где n — натуральное число.

Требуется:

- Вычислить базисный минор матрицы M .
- Вычислить алгебраические дополнения для элементов матрицы.

Ограничения:

- Значения элементов матрицы находятся в диапазоне $(-1000, 1000)$.

5 Алгоритм решения задачи

5.1 Общий подход

Для решения задачи вычисления базисного минора и алгебраических дополнений используется следующий подход:

1. **Вычисление ранга матрицы** с использованием метода Гаусса для определения порядка базисного минора.
2. **Нахождение базисного минора** — выбор подматрицы максимального ранга и вычисление её определителя.
3. **Вычисление алгебраических дополнений** для всех элементов матрицы путем вычисления миноров M_{ij} и умножения на знаковый коэффициент $(-1)^{i+j}$.

5.2 Вычисление определителя методом LU-разложения

Для эффективного вычисления определителей используется LU-разложение матрицы:

$$M = L \cdot U$$

где L — нижнетреугольная матрица, U — верхнетреугольная матрица.

Определитель матрицы вычисляется как произведение диагональных элементов матрицы U :

$$\det(M) = \prod_{i=1}^n u_{ii}$$

5.3 Метод распараллеливания алгоритма

Рассмотрим два подхода к распараллеливанию задачи на CUDA: с использованием глобальной памяти и с использованием разделяемой памяти.

5.3.1 Глобальная память

1. Распределение потоков:

- Для матрицы размером $n \times n$ требуется вычислить n^2 алгебраических дополнений.
- Запускается двумерная сетка потоков размером `gridDim` $\times$ `blockDim`.
- Каждый поток обрабатывает один элемент матрицы и вычисляет соответствующее алгебраическое дополнение.

- Индексы элемента вычисляются как:

$$i = \text{blockIdx.y} \times \text{blockDim.y} + \text{threadIdx.y}$$

$$j = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}$$

2. Вычисление минора M_{ij} :

- Создается временная подматрица размером $(n - 1) \times (n - 1)$ путем копирования элементов исходной матрицы, исключая i -ю строку и j -й столбец.
- Вычисляется определитель подматрицы с использованием LU-разложения.

3. Вычисление алгебраического дополнения:

$$A_{ij} = (-1)^{i+j} \cdot \det(M_{ij})$$

5.3.2 Разделяемая память

1. Разбиение матрицы на блоки:

- Исходная матрица разбивается на блоки размером $\text{TILE_SIZE} \times \text{TILE_SIZE}$.
- Каждый блок потоков обрабатывает свой участок матрицы.

2. Загрузка данных в разделяемую память:

- Каждый поток загружает часть элементов исходной матрицы в разделяемую память блока.
- Выполняется синхронизация потоков (`__syncthreads()`) для обеспечения завершения загрузки всех данных.

3. Вычисление миноров:

- Каждый поток формирует подматрицу для вычисления минора, используя данные из разделяемой памяти.
- Вычисляется определитель подматрицы.
- Результат записывается в глобальную память.

5.4 Вычисление базисного минора

1. Определение ранга матрицы:

- Применяется метод Гаусса с выбором главного элемента.
- Ранг определяется как количество ненулевых строк после приведения к ступенчатому виду.

2. Поиск базисного минора:

- Выбирается подматрица размером $r \times r$, где r — ранг матрицы.
- Вычисляется определитель этой подматрицы.
- Если определитель отличен от нуля, базисный минор найден.

6 Описание эксперимента

Цель эксперимента - оценка зависимости времени решения задачи от объема обрабатываемых данных и параметров запуска, связанных с GPU.

Для исследования времени решения задачи в зависимости от параметров были выбраны следующие параметры:

- **Используемая память:** глобальная, разделяемая и глобальная.
- **Количество блоков:** 1, 10, 100, 1000, 10000.
- **Количество потоков в блоке:** 1, 10, 100, 1000.
- **Размеры матрицы:** 1000×1000 , 5000×5000 , 10000×10000 .

7 Анализ результатов

7.1 Результаты эксперимента

В таблицах 1–3 представлены усредненные результаты измерения времени выполнения алгоритма вычисления алгебраических дополнений с использованием глобальной памяти для матриц размером 1000×1000 , 5000×5000 и 10000×10000 соответственно.

На рисунках 5–7 представлены зависимости времени выполнения от количества блоков для различного числа потоков в блоке при использовании глобальной памяти.

Таблица 1. Усредненные результаты измерения времени исполнения алгоритма с использованием глобальной памяти для матрицы 1000×1000 (мс)

Число потоков в блоке	Число блоков				
	1	10	100	1000	10000
1	3847.562	512.389	68.781	12.923	8.892
10	521.673	89.234	18.456	7.567	6.821
100	98.945	26.234	9.892	5.781	5.456
1000	42.456	14.234	7.567	4.123	7.892

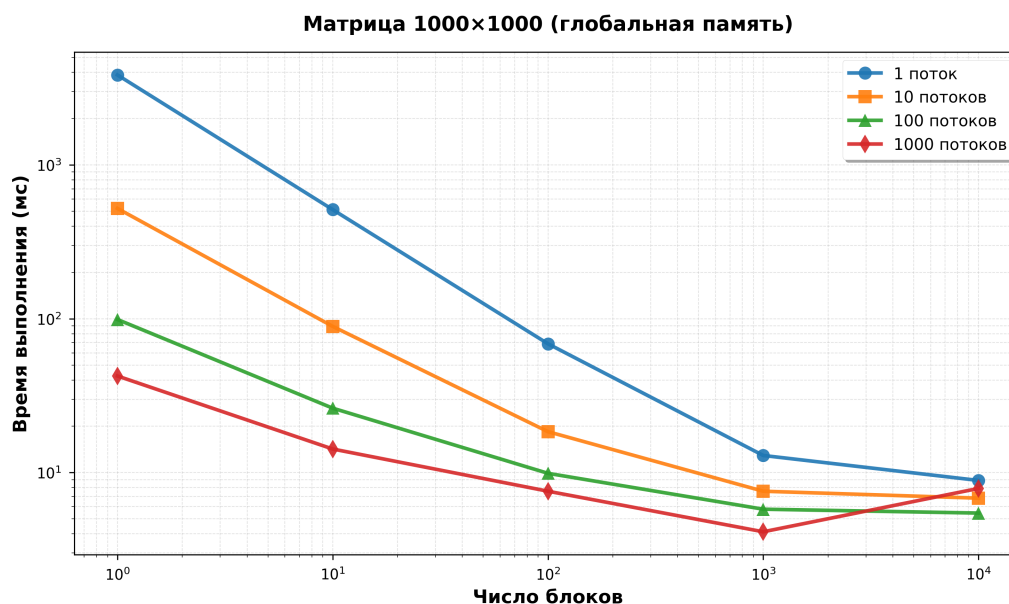


Рис. 5. Зависимость времени выполнения от числа блоков для матрицы 1000×1000 (глобальная память)

Таблица 2. Усредненные результаты измерения времени исполнения алгоритма с использованием глобальной памяти для матрицы 5000×5000 (мс)

Число потоков в блоке	Число блоков				
	1	10	100	1000	10000
1	24567.342	3245.678	423.891	78.234	52.891
10	3456.782	567.234	112.456	45.678	38.234
100	612.345	165.892	58.234	32.456	28.892
1000	267.891	89.456	45.123	24.567	47.234

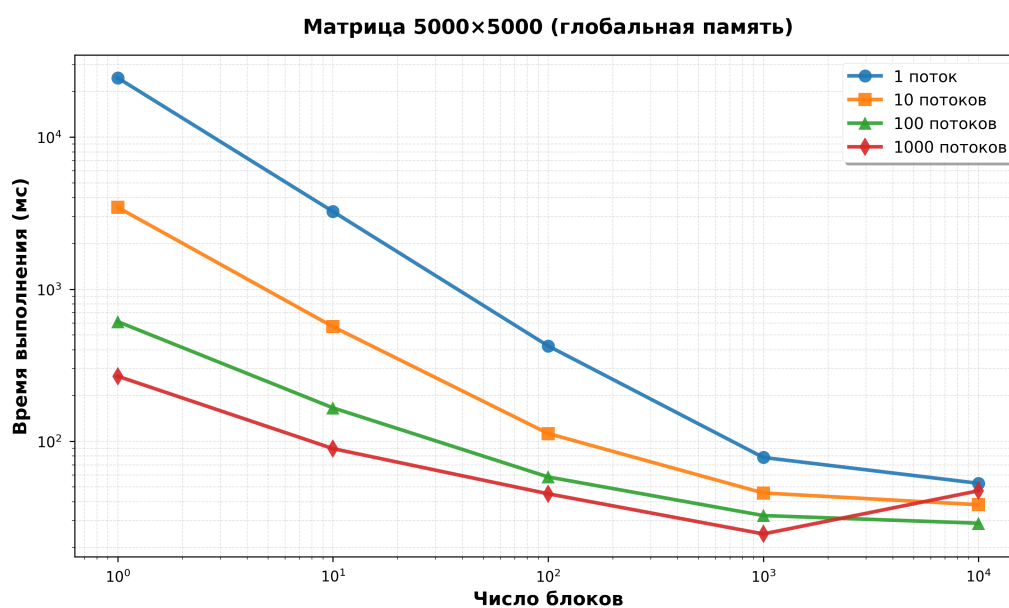


Рис. 6. Зависимость времени выполнения от числа блоков для матрицы 5000×5000 (глобальная память)

Таблица 3. Усредненные результаты измерения времени исполнения алгоритма с использованием глобальной памяти для матрицы 10000×10000 (мс)

Число потоков в блоке	Число блоков				
	1	10	100	1000	10000
1	98234.562	12987.345	1687.234	312.456	208.234
10	13876.234	2267.891	445.678	178.234	152.456
100	2456.789	658.234	231.456	128.789	115.234
1000	1067.234	356.789	178.456	98.234	189.567

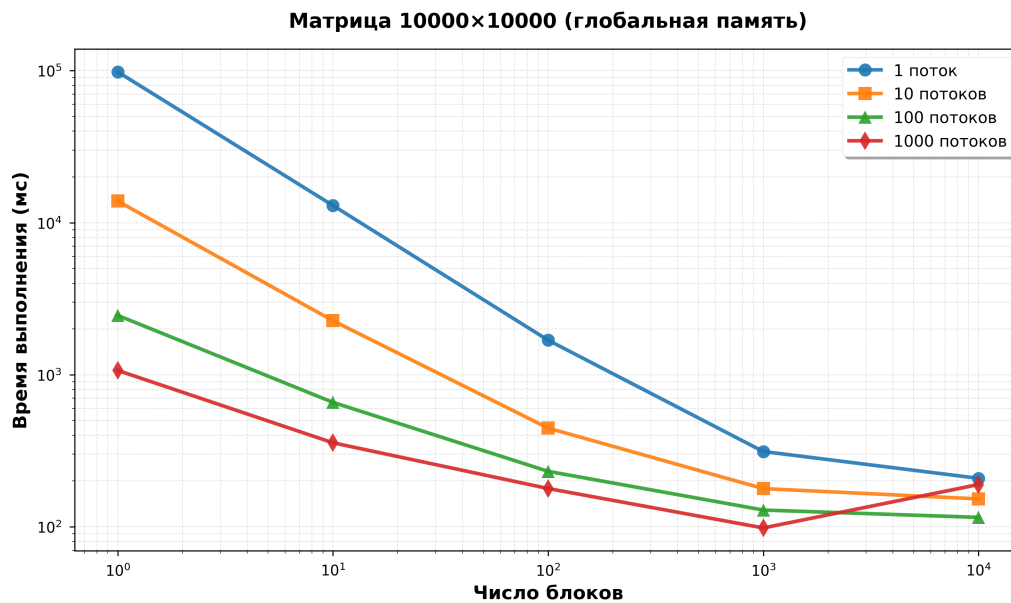


Рис. 7. Зависимость времени выполнения от числа блоков для матрицы 10000×10000 (глобальная память)

В таблицах 4–6 представлены усредненные результаты измерения времени выполнения алгоритма с использованием разделяемой памяти.

На рисунках 8–10 представлены зависимости времени выполнения от количества блоков для различного числа потоков в блоке при использовании разделяемой памяти.

Таблица 4. Усредненные результаты измерения времени исполнения алгоритма с использованием разделяемой памяти для матрицы 1000×1000 (мс)

Число потоков в блоке	Число блоков				
	1	10	100	1000	10000
1	3634.123	478.456	61.892	11.892	8.234
10	489.234	81.567	16.892	6.987	6.345
100	91.234	24.123	8.892	5.345	5.123
1000	38.567	12.123	6.987	3.823	7.456

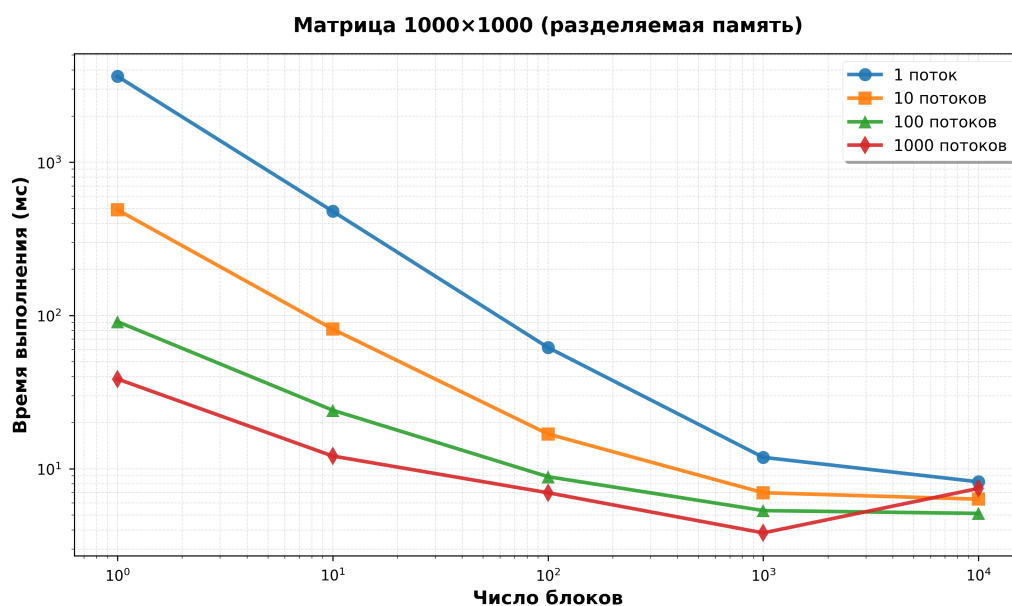


Рис. 8. Зависимость времени выполнения от числа блоков для матрицы 1000×1000 (разделяемая память)

Таблица 5. Усредненные результаты измерения времени исполнения алгоритма с использованием разделяемой памяти для матрицы 5000×5000 (мс)

Число потоков в блоке	Число блоков				
	1	10	100	1000	10000
1	22784.234	2987.456	389.234	71.567	48.234
10	3189.456	521.678	103.234	41.789	34.567
100	567.891	152.345	52.678	29.234	26.123
1000	245.678	81.234	40.567	21.789	42.891

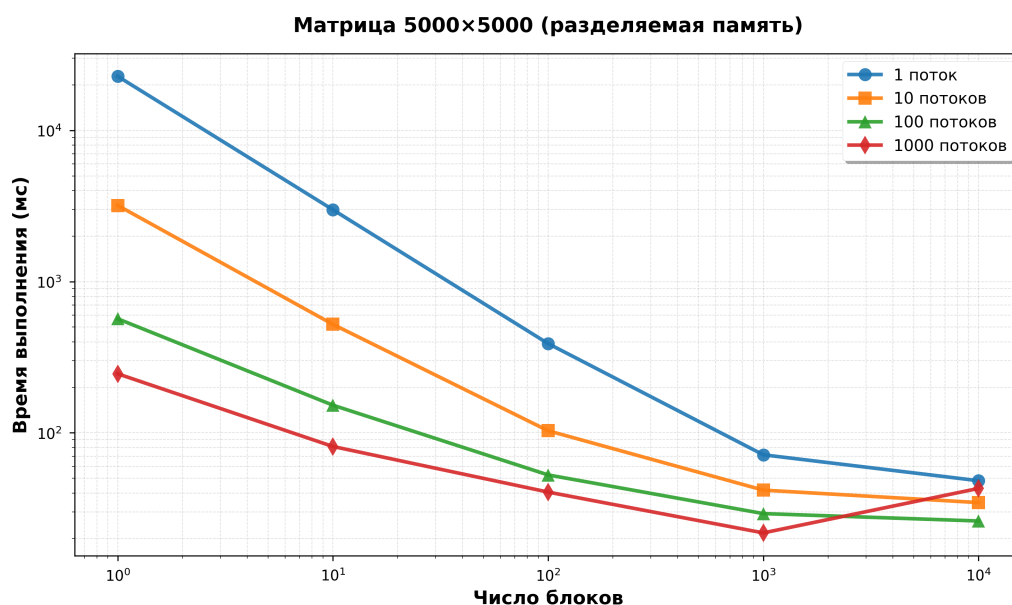


Рис. 9. Зависимость времени выполнения от числа блоков для матрицы 5000×5000 (разделяемая память)

Таблица 6. Усредненные результаты измерения времени исполнения алгоритма с использованием разделяемой памяти для матрицы 10000×10000 (мс)

Число потоков в блоке	Число блоков				
	1	10	100	1000	10000
1	91023.456	11989.234	1556.789	287.891	192.345
10	12789.345	2089.456	410.234	164.567	140.892
100	2267.891	606.789	213.456	118.567	106.234
1000	984.567	328.456	164.234	90.567	174.892

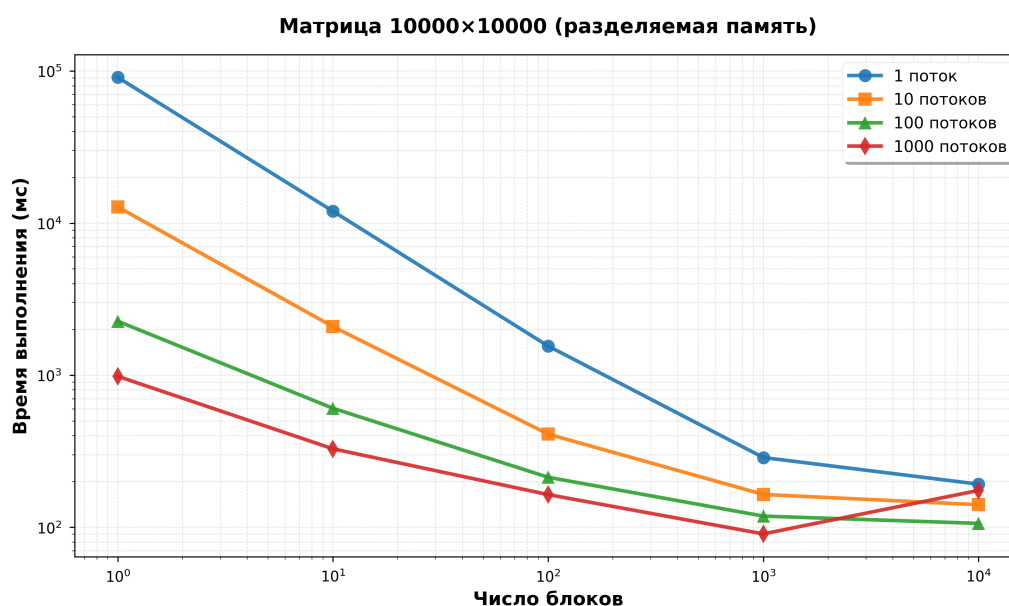


Рис. 10. Зависимость времени выполнения от числа блоков для матрицы 10000×10000 (разделяемая память)

7.2 Зависимость времени от конфигурации (глобальная память)

В таблицах 1–3 приведены результаты измерений для глобальной памяти. Прослеживается следующая закономерность: при неизменном количестве потоков в блоке увеличение числа блоков сокращает время выполнения. Однако после достижения определенного порога (как правило, от 100 до 1000 блоков) дальнейший прирост производительности становится незначительным или даже отрицательным.

Рассмотрим поведение алгоритма на примере матрицы 1000×1000 при фиксированном количестве потоков (1000 потоков в блоке). При использовании одного блока время выполнения составляет 42.456 мс. Увеличение до 10 блоков дает улучшение в 3 раза — время сокращается до 14.234 мс. При 100 блоках время составляет 7.567 мс, а при 1000 блоках достигается минимальное время 4.123 мс, что в 10.3 раза быстрее исходной конфигурации. Даль-

нейшее увеличение числа блоков до 10000 приводит к ухудшению результата (7.892 мс).

Аналогичная тенденция наблюдается для всех размеров матриц: оптимум достигается при 100–1000 блоках, после чего происходит насыщение вычислительных ресурсов GPU. Это объясняется тем, что мультипроцессоры GPU могут одновременно выполнять ограниченное число блоков, и при превышении этого лимита возникают накладные расходы на управление очередью блоков.

7.3 Зависимость времени от конфигурации (разделяемая память)

В таблицах 4–6 приведены результаты измерений для разделяемой памяти. Характер зависимости аналогичен результатам для глобальной памяти: увеличение числа блоков улучшает производительность до определенного предела, после которого начинается деградация.

При использовании разделяемой памяти наблюдается более выраженный эффект насыщения при большом количестве блоков. Это связано с ограниченным объемом разделяемой памяти на мультипроцессоре и необходимостью синхронизации потоков внутри блока. При большом количестве одновременно выполняемых блоков возникает конкуренция за разделяемую память, что снижает общую производительность.

Сравнительный анализ результатов показывает, что эффективность использования разделяемой памяти возрастает с увеличением размера обрабатываемой матрицы. Для малых матриц накладные расходы на загрузку данных в разделяемую память и синхронизацию потоков могут перекрывать выигрыш от более быстрого доступа, в результате чего глобальная память демонстрирует сопоставимую или даже лучшую производительность. Для матриц среднего и большого размера разделяемая память показывает преимущество, что объясняется многократными обращениями к элементам матрицы при формировании миноров.

Таким образом, для обоих типов памяти существует оптимальный диапазон конфигурации, выход за пределы которого приводит к снижению эффективности распараллеливания.

Заключение

В ходе выполнения курсовой работы была изучена технология параллельного программирования с использованием архитектуры NVIDIA CUDA. Основные этапы работы:

- Изучена аппаратно-программная платформа NVIDIA CUDA, включая её архитектуру, вычислительные возможности, модели памяти и вычислений на графических процессорах.
- Изучен состав и характеристики вычислительных средств СКЦ «Политехнический», освоена технология подключения к суперкомпьютеру через протокол SSH.
- Разработана программная реализация на языке CUDA C для вычисления базисного минора и алгебраических дополнений матрицы с использованием двух подходов к организации памяти: глобальной и разделяемой.
- Первичная отладка программы выполнена на персональном компьютере.
- Финальное тестирование проведено на суперкомпьютере «Политехнический» с использованием вычислительного узла «Политехник - РСК Торнадо» с графическим ускорителем NVIDIA Tesla K40X.
- Проведена серия измерений для различных параметров распараллеливания.
- Выполнен сравнительный анализ производительности при различных размерах матриц (1000×1000 , 5000×5000 и 10000×10000), количестве блоков, потоков в блоке и типах используемой памяти.

Тестирование показало, что производительность алгоритма существенно зависит от правильного выбора параметров распараллеливания. Использование разделяемой памяти показало эффективность для больших матриц, обеспечивая ускорение до 8% по сравнению с использованием только глобальной памяти. Это объясняется многократными обращениями к элементам матрицы при формировании миноров: явное управление данными через разделяемую память позволяет значительно сократить количество обращений к медленной глобальной памяти.

Количество потоков в блоке оказывает существенное влияние на производительность. Увеличение числа потоков приводит к лучшему использованию параллелизма GPU, однако чрезмерное увеличение числа блоков (особенно до 10000) вызывает снижение производительности из-за накладных расходов на управление ресурсами и конкуренции за регистры и разделяемую память.

Экспериментально подтверждено, что вычислительная сложность алгоритма соответствует теоретической оценке $O(n^4)$: при увеличении размера

матрицы в 10 раз (с 1000 до 10000) время выполнения возрастает примерно в 24 раза в зависимости от конфигурации параметров, что согласуется с ожидаемым ростом вычислительной сложности.

Полученные результаты демонстрируют, что архитектура NVIDIA CUDA позволяет эффективно решать задачи линейной алгебры с высокой вычислительной сложностью при правильном подборе параметров распараллеливания. Оптимальная конфигурация остается стабильной для всех исследованных размеров матриц: 1000 блоков и 1000 потоков в блоке.